

Group 6: Abstract, Introduction, Conclusion

Jun Tang
Fairleigh Dickinson University
Vancouver, Canada
j.tang4@student.fdu.edu

Xiaoyi Han
Fairleigh Dickinson University
Vancouver, Canada
x.han1@student.fdu.edu

Kaiwen Wang
Fairleigh Dickinson University
Vancouver, Canada
k.wang4@student.fdu.edu

Ziyan Qiu
Fairleigh Dickinson University
Vancouver, Canada
z.qiu1@student.fdu.edu

Hongyi Niu
Fairleigh Dickinson University
Vancouver, Canada
h.niu@student.fdu.edu

1 Related Work

Many papers have been published on flash caching strategies for storage systems. We searched the peer-reviewed work from 2019-2025 in FAST, OSDI, NSDI, SOSP, EuroSys, USENIX ATC, ICS, MSST, and SIGMETRICS, which are related to flash caching, admission, prefetching strategies and/or policies, especially guided by machine learning methods, and cost saving for flash write control. For each paper we read, we map to Baleen’s main ideas: peak Disk-head Time(D), episode-based supervision/OPT imitation, admission-prefetch coordination, and endurance/TCO modeling. From the papers found, We can summarize prior research into three main buckets: (i) flash caching for HDD backends, (ii) admission and prefetching strategies, including machine learning (ML)-guided methods, and (iii) cost/endurance-aware control of flash write rates (TCO). We summarized the works for each of these bucket and their pros and cons. After that, we provide a taxonomy matrix and a timeline that evidences the progression from heuristic policies to ML-guided admission/eviction/prefetch. We exclude those work on client-side/CDN caches, CPU/GPU cache prefetching, and non-flash tiering.

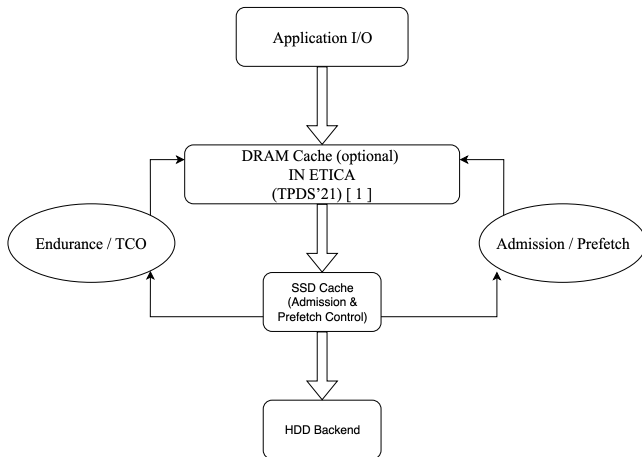


Figure 1: Architecture of HDD-backed SSD caching systems in Bucket A.

1.1 Bucket A: FlashCaching for HDD Backends

Bucket A Summary (HDD-backed Flash Caching). This bucket studies systems that interpose an SSD cache in front of HDD backends and pursue high hit rate and low latency using lightweight, deployment-friendly mechanisms. Early tiered designs such as *FlashTier* treat the SSD as an OS-managed cache/tier with durable placement and crash consistency, making “flash-as-cache” practical in production while closing most of the DRAM–HDD gap for common OLTP and KV-style workloads. Subsequent work improves device efficiency by restructuring I/O: *RIPQ* groups related objects and writes them sequentially to curb random updates, increasing throughput and extending device lifetime; *LARC*-style lazy/adaptive admission filters out short-lived blocks and avoids needless flash writes while preserving simple replacement. Across these systems, prefetch—when present at all—is typically heuristic and decoupled from admission, and endurance is handled implicitly via write-amplification reduction rather than by an explicit write-rate budget tied to service constraints at the HDD layer. Figure 1 illustrates the common control loop (DRAM → SSD → HDD) and Algorithm 1 summarizes representative admission/prefetch logic for this family.

From a design-space view, Bucket A optimizes steady-state latency and average hit rate with minimal metadata and modest online computation, favoring robust policies that can tolerate workload churn without per-object modeling. The drawback is that these policies rarely elevate *peak* backend load to a first-class objective: under bursty traffic, HDD queues can saturate even if average hit rate is high. Moreover, because admission and prefetch are tuned independently, aggressive prefetch can pollute the cache and waste write budget, while conservative admission may starve beneficial prefetch. *In contrast to Baleen*, our capstone baseline explicitly targets peak Disk-head Time (DT) and co-trains admission with coordinated prefetch under a TCO/endurance-aware controller using episode/OPT-style supervision, turning endurance limits and backend service capacity into explicit constraints rather than side effects.

Figure 1 shows the architecture, and Algorithm 1 summarizes the control logic.

As shown in Figure 1 and Algorithm 1, HDD-backed SSD caching focuses on hit rate and latency. *In contrast to Baleen*, this line of work does not set the peak disk-head time (DT) as a primary objective and does not co-train admission with coordinated prefetch under a TCO-aware controller.

Pros.

Algorithm 1 Admission and Prefetch Control for HDD-backed SSD Caching

```
1: for each I/O request req do
2:   if DRAM_cache.hit(req) then
3:     Serve from DRAM
4:   else if SSD_cache.hit(req) then
5:     Serve from SSD
6:     Update reuse statistics
7:   else
8:     Fetch from HDD
9:     if Admission_Policy(req) == TRUE then    ▷ based on
10:      frequency / reuse
11:      Place req in SSD_cache
12:   if Prefetch_Policy(req) == TRUE then      ▷ decoupled
13:     Prefetch sequential or related blocks to SSD
14:   Monitor SSD endurance and TCO
```

(1) **High Hit-Rate and Latency Reduction:** Across all ten papers, SSD caching for HDD backends consistently improves hit rate and reduces end-to-end request latency, providing a cost-effective way to close the performance gap between HDD and DRAM. Early tiering designs such as FlashTier [?] demonstrate that treating SSDs as an OS-managed cache layer yields robust, crash-consistent deployments with significant latency improvements. Later work such as RIPQ [?] and SeqPack [?] further focus on write layout and sequentialization, minimizing random I/O patterns while sustaining high throughput. Multi-level caches like ETICA [?] combine DRAM and SSD caching tiers to hide I/O stalls in virtualized workloads, providing both higher hit ratios and improved tail-latency guarantees. Together, these studies form strong empirical evidence that SSD caches are an effective, scalable mechanism to reduce average response time even for read-heavy, bursty, or mixed workloads.

(2) **Write-Amplification Mitigation and Endurance Awareness:** A second recurring strength is the explicit attention to SSD endurance and write-amplification mitigation. LARC [?] introduces lazy/adaptive admission to filter out transient data, avoiding unnecessary flash writes. FairyWREN [?] goes further, co-designing garbage collection and admission under write-budget constraints to make caching sustainable over device lifetime. Recent systems such as FDP-based CacheLib [?] exploit NVMe Flexible Data Placement to physically isolate cache traffic and reduce device-level garbage-collection overhead. These optimizations not only prolong device life but also reduce total cost of ownership (TCO) and embodied carbon footprint—an increasingly important consideration for datacenter operators.

Cons.

(1) **Lack of Peak-Load Awareness and DT Minimization:** Most HDD-backed flash caching systems are primarily optimized for average-case hit rate and latency, but rarely treat peak disk-head time (DT) or queuing delay as a first-class optimization target. Under bursty or adversarial workloads, this can lead to backend saturation and elevated tail latencies. For example, while SeqPack [?] improves sequential write batching, it still lacks an explicit

mechanism to throttle cache admission to respect HDD service-rate constraints. Without DT-aware admission or feedback loops, these systems cannot guarantee predictable service under peak load, which is critical for SLAs in production.

(2) **Limited Prefetch Coordination, Cost Modeling, and Global Control:** I Prefetching—when implemented at all—is typically orthogonal to admission decisions and therefore prone to cache pollution, evicting useful data and generating unnecessary flash writes. None of the classical HDD-backed systems integrate prefetch and admission under a single optimization objective. Moreover, explicit modeling of flash write budgets, dollar cost, or energy consumption is mostly absent; endurance handling is heuristic rather than globally optimized. This lack of joint cost modeling leaves room for inefficiencies, especially as SSD price/performance characteristics and workload mixes evolve. Compared to Baleen [?], which co-trains ML-based admission and prefetch policies under a TCO/DT-aware objective, these earlier systems represent a more ad hoc and uncoordinated design space.

1.2 Bucket B: ML-Guided Caching/Prefetching

In contrast to Baleen, although prior work has some similar ideas in grouping access and using ML-guided strategies, they either focus on eviction decisions (GL-Cache [?]) or admission under cost/endurance constraints (CacheSack [?]).

ML-guided caching targeting per-object replacement decisions has low efficiency, as maintaining and training models at the object granularity is low performance and fail to adapt to large-scale system with variant access patterns. In GL-Cache [?], author stated that in object-level learned caches (e.g., LRB), each eviction must sample dozens of objects and run inference per write, because predictions rely on dynamic per-object features whose values change. This makes the prediction can not be reused eventually. In their test, LRB sampled 32 objects per eviction, which spent about 200 μ s per eviction on one CPU core. This make it capped at around 5,000 evictions/s per core, which is far below the requirement of a production server that is $> 100,000$ req/s. Yang et al., in GL-Cache [?], address this limitation by introducing group-level learning. GL-Cache [?] empirically uses write-time to group the objects and place them into a fixed-size group. They justified that objects written at the same time tend to have similar reuse behavior. Statistically, the dispersion of mean reuse time inside a write-time group is tighter than other ways to group.

They define object utility as

$$U_o(t) = \frac{1}{T_{\text{next},o}(t) s_o},$$

and group utility as the sum across objects,

$$U_{\text{group}}(t) = \sum_{o \in G} U_o(t).$$

Then GL-Cache learns a function $F(X_{\text{group}}) \rightarrow U_{\text{group}}$ using gradient-boosted trees with 7 features: static features(request rate, write rate, miss ratio, mean object size) and dynamic ones(group age, number of requests, number of requested objects). By the model predicting, GL-Cache evicts the least-use group (minimum-utility group).

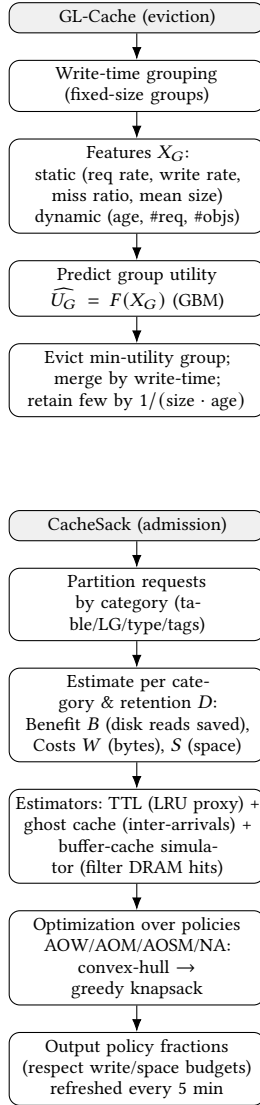


Figure 2: Bucket B (ML-guided caching): GL-Cache predicts group utility to drive eviction; CacheSack estimates benefit/cost and optimizes admission via a knapsack procedure.

They evaluated 118 traces across CloudPhysics (103 VM block-I/O), MSR (14 disk-I/O), and Wikimedia CDN (1 web trace), measuring hit ratio and throughput [?]. GL-Cache achieves hit-ratio gains of +7% on average and +25% at P90 vs LRB, and +3% on average / +13% at P90 vs the best learned baseline [?]. Throughput-wise, GL-Cache-E is 228 $\times$ and GL-Cache-T 586 $\times$ faster than LRB at small cache sizes, and about +64% faster than the fastest learned cache overall; the prototype meets production-level throughput [?].

The prior cache admission policy focused mainly on maximizing the hit rate, which tends to cache everything hit even when the cache brings little performance merit. This will require massive flash writes that hurt SSD lifetime. Wei et al., in CacheSack [?], proposed a trained model to predict the utility of admitting each block: they partition requests into categories (e.g., table/locality

group/type or user annotations) and, for each category, choose a mixture over four simple admission policies (AdmitOnWrite, AdmitOnMiss, AdmitOnSecondMiss/LARC, NeverAdmit). CacheSack doesn't use hit-rate as the target. For each traffic category and modeled retention time D , it estimates how many disk reads would be avoided (benefit) and how many flash bytes and flash space would be consumed (costs).

$$\begin{aligned}
 B_{c,p}(D) &: \text{disk reads saved (benefit),} \\
 W_{c,p}(D) &: \text{bytes written to flash,} \\
 S_{c,p}(D) &: \text{cache occupancy,} \\
 U_{c,p}(D) &= \beta B_{c,p}(D) - \gamma W_{c,p}(D) - \delta S_{c,p}(D).
 \end{aligned}$$

Fix D and let i index the pairs (c, p) . Then

$$\max_{\{\alpha_i\}} \sum_i \alpha_i U_i \quad \text{s.t.} \quad \sum_i \alpha_i W_i \leq W_{\max}, \quad \sum_i \alpha_i S_i \leq S_{\max}.$$

They model retention with TTL (an LRU proxy) and log request gaps in a ghost cache. And CacheSack estimates per-category benefit (disk reads saved) and costs (bytes written to flash and cache occupancy) using the TTL in the ghost cache and lightweight buffer-cache simulators to reflect the real disk avoidance rather than DRAM buffer hits. It then formulates admission as a cost-benefit optimization across modeled retention times and solves it efficiently via a convex-hull + greedy knapsack procedure (refer to Algorithm 2). By doing so, they produce policy fractions respecting cache-usage/write budget and refresh every five minutes. In Google production (Colossus Flash Cache), CacheSack became the default admission policy and, compared to LARC, it reduces disk reads by about 6%, cut flash bytes written by about 26%, and improved overall TCO by about 6.5% while sustaining backend performance at scale.

Relative to Baleen's ML-guided admission and prefetching based on episode, GL-Cache contributed ML-guided eviction at group granularity, while CacheSack contributed ML-guided, cost-aware admission; neither coordinates admission with prefetch under a DT objective.

Pros.

(1) GL-Cache applies ML-guided models to make eviction decisions, whereas Baleen relies on a fixed FIFO eviction policy. By clustering objects with similar access patterns into groups, GL-Cache trains predictive models at the group level to rank groups by utility and selectively evict the least useful ones. It ranks groups by a learned group-utility that favors soon-to-be-reused, small objects, then evicts the minimum-utility group while preserving a few best objects via a cheap $1/(\text{size} \cdot \text{age})$ rule. At the same time, inference is amortized—GL-Cache reuses a single group ranking across many evictions, retraining daily (≈ 10 –50 ms) with per-inference latency ≈ 0.4 –3 ms and ~ 5 B/object metadata, so eviction stays lightweight at scale. This design allows GL-Cache to adaptively predict future reuse and tailor eviction across workloads, which improves hit ratios and maintains throughput even at scale.

Algorithm 2 Bucket B: Category-level ML-guided admission selection (CacheSack)

Require: Categories C ; candidate policies $\mathcal{P}$ (AOM, AOSM, AOW, NA, fractional); flash write budget B

```

1: for each  $c \in C$  do
2:   for each  $p \in \mathcal{P}$  do
3:      $\text{benefit}[c, p] \leftarrow \widehat{\Delta\text{reads}}(c, p)$   $\triangleright$  ML-predicted disk I/O saved
4:      $\text{cost}[c, p] \leftarrow \widehat{\text{bytes}}(c, p) + \lambda \cdot \widehat{\text{occupancy}}(c, p)$ 
5:      $\text{ratio}[c, p] \leftarrow \text{benefit}[c, p] / \text{cost}[c, p]$ 
6:      $p^*(c) \leftarrow \arg \max_{p \in \mathcal{P}} \text{ratio}[c, p]$ 
7: Sort pairs  $(c, p^*(c))$  by  $\text{ratio}[c, p^*(c)]$  in descending order
8:  $\beta \leftarrow 0$   $\triangleright$  used flash-write budget
9: for each  $(c, p^*(c))$  in order do
10:  if  $\beta + \text{cost}[c, p^*(c)] \leq B$  then
11:     $\alpha(c) \leftarrow 1$ ;  $\beta \leftarrow \beta + \text{cost}[c, p^*(c)]$ 
12:  else
13:     $\alpha(c) \leftarrow \max(0, \min(1, \frac{B-\beta}{\text{cost}[c, p^*(c)]}))$ 
14:  break
15: return  $\{(\alpha(c), p^*(c)) \mid c \in C\}$ 

```

(2) CacheSack’s admission model accepts multiple dimensions: disk-read, flash write, and cache usage. Based on these dimensions, CacheSack models admission as a cost–benefit optimization. For each traffic category, it estimates the benefit(disk I/O saved), and costs which include parameters of bytes written(SSD endurance hit) and cache consumption. Then it formulates the optimization as a knapsack problem to pick the optimal admission decision per category (AdmitOnMiss, AdmitOnSecondMiss, AdmitOnWrite, NeverAdmit, or fractional combinations). For each category and a grid of retention times D , it estimates benefit B (disk reads saved) and costs (W, S) (bytes written, space), prunes dominated (cost, benefit) points via a convex-hull step, and runs a greedy fractional knapsack to choose policy fractions $\alpha_{c,p}(D)$ under write/space budgets. The selected fractions are applied online after the next 5-minute retrain, steering admission. By this ML-guided policy making, it minimizes the overall cost.

Cons.

(1) GL-Cache focuses only on eviction policies. While its group-level learning improves scalability and hit ratios, it does not address admission or prefetching. This makes it only a solution for one aspect of the cache optimization problem, instead of one for a complete caching framework. It provides no endurance or TCO modeling, and ignores system-level peak-load DT objectives. In addition, write-time grouping is fixed; workload shifts can misalign groups and invalidate the prediction. Its once-per-day retraining cadence can lag fast diurnal shifts, reducing responsiveness to sudden traffic changes.

(2) CacheSack categorizes requests into five admission policies (AdmitOnMiss, AdmitOnSecondMiss, AdmitOnWrite, NeverAdmit, and fractional combinations). Its model is closely coupled to Google’s datacenter flash cache environment and highly relies on workload-specific partitioning to estimate costs and benefits. This tight coupling limits the generality of the approach, making it less portable to other caching systems or workloads. It also depends on

DRAM-buffer simulators and a TTL–LRU approximation; misestimation there can skew the benefit/cost predictions.

1.3 Bucket C: Cost/Endurance/TCO

In contrast to Baleen, past work in this bucket mainly controls how many bytes go to flash. It does not try to lower peak backend load (DT) and do coordinated prefetch at the same time. We use two works: *Flashshield* (NSDI’19) and *FairyWREN* (OSDI’24). Flashshield cuts writes with simple ML admission and with in-order writes on SSD [?]. FairyWREN treats the write rate like a budget and uses ZNS/FDP so admission and garbage collection work together [?]. These works show that write amplification and device life must be key parts of admission. *In contrast to Baleen, they do not set DT as a goal or pair admission with OPT-guided, coordinated prefetch.*

We use a simple way to think about cost and endurance. The flash write rate is something we can control. We set a limit, called $W_{\max}$. If we stay under this limit, the flash device will last longer. We also use a small formula to show total cost:

$$\text{TCO} = a \cdot \text{Reads} + b \cdot \text{Writes} + c \cdot \text{Capacity}.$$

Here, “Reads” means disk reads saved by the cache. “Writes” means data written to flash. “Capacity” means the space used in the flash. If we write too much, the flash wears out. So we want to keep writes low. Both Flashshield and FairyWREN try to write less. They also check if the hit rate is still good. These systems do not try to lower DT, but they help cost and flash life.

This algorithm has two parts. First, it learns which objects are worth storing in flash. Then, it limits how often flash is written. Flashshield does this using simple filters and sequential writes. FairyWREN does this by working with the flash device itself. These systems help reduce writes and improve lifetime. *In contrast to Baleen, they do not learn from access traces to optimize backend load or prefetch.*

Flashshield and FairyWREN both try to lower the total cost of using flash by writing less. They do this in different ways. Flashshield looks at each object and filters out the ones not worth storing in flash. FairyWREN focuses on how to write data in a smarter way by using new flash features. Both systems help lower write amplification. This reduces how fast the flash wears out. When flash lasts longer, the cost per year is lower. This helps data centers save money over time. These ideas support the goal in Baleen to choose a good write rate, instead of using a fixed one.

As flash devices become more common in data centers, their cost and lifetime will matter even more. If a system writes too much, the flash will wear out faster. This means it must be replaced sooner, which increases cost. Also, replacing flash often creates waste and uses more energy. A good caching system should try to reduce this. It should choose when and what to write based on cost, lifetime, and system goals. Flashshield and FairyWREN show two ways to do this. Baleen goes further by using learning and supervision to pick a good write rate while also handling peak backend load.

Pros.

(1) Flashshield uses a simple ML model and filters out bad objects. It also writes data in order, which helps the device. This lowers write overhead and keeps hit rate high [?].

(2) FairyWREN works with new flash hardware. It writes less and extends flash life. It also saves cost and energy in long-term use [?].

Cons.

(1) Flashield only works well for key-value workloads. It does not model cost or try to reduce backend load [?].

(2) FairyWREN needs special flash hardware (like ZNS). It does not use machine learning or coordinate prefetch [?].

Algorithm 3 Bucket C: Endurance/TCO-aware admission (simple sketch)

Require: Request stream; features x ; predictor $\widehat{S}(x)$ for saved disk reads; current write rate w ; write budget $W_{\max}$; DLWA estimate $\widehat{\alpha}$; costs $\lambda_{\text{disk}}, \lambda_{\text{flash}}, \lambda_{\text{cap}}$.

- 1: **for** each miss on object o **do**
- 2: score: $b = \widehat{S}(x) \cdot \lambda_{\text{disk}}, c = \lambda_{\text{flash}}$
- 3: flash-worthiness (Flashield): if (size(o), hotness(o)) say “yes”, keep; else stay in DRAM
- 4: rate guard: admit iff $b/c \geq \tau$ and keep $w \cdot \widehat{\alpha} \leq W_{\max}$
- 5: placement: write in order (Flashield) or pack with GC on ZNS/FDP (FairyWREN)
- 6: periodic step: tune τ to lower $\text{TCO} = \lambda_{\text{disk}} \cdot \text{Reads} + \lambda_{\text{flash}} \cdot (w \cdot \widehat{\alpha}) + \lambda_{\text{cap}} \cdot \text{FlashCap}$ while still $w \cdot \widehat{\alpha} \leq W_{\max}$

1.4 Taxonomy Matrix

Table 1 compares the surveyed articles across five aspects—granularity, objective, prefetch coordination, supervision, and endurance model—using concise Baleen-based tags. The taxonomy matrix shows a clear shift from heuristic block/segment designs to ML/analytical approaches. In recent work, TCO/endurance objectives have emerged as a focus. Prefetch is common in segment-centric systems (NVMe Flexible SSDs, SeqPack, ETICA, Multi-Cache, TLC-Cache, HybridStore), while others omit it; among them, Baleen uniquely coordinates admission+prefetch under a DT objective. Supervision patterns also vary widely—ranging from heuristic and analytical estimation to supervised ML and RL—while OPT-imitation (as in Baleen) appears only rarely.

1.5 Timeline and Evidence

Table 2 groups prior works into three periods and shows how the focus shifts over time. The research focus have shifted from hit rate/latency to admission filtering to balance the cost of flash, and recently to cost/TCO-ware and ML guided. Prefetch also evolved: it was mostly absent or decoupled in earlier periods; Baleen coordinates it with admission under a DT objective, while most other systems still treat it separately. In addition, recent work introduces interface-level mechanisms (e.g., NVMe FDP and GC co-design) to curb write amplification and tail latency, complementing ML-based policies.

Table 1: Taxonomy matrix of ML-guided caching/prefetching papers.

Paper	Gran.	Objective	Prefetch	Supervision	Endurance
NVMe Flexible SSDs (EuroSys’25) [?]	Block / Segment	Latency / TCO	Yes	Heuristic	Empirical
Baleen (FAST’24) [?]	Segment	DT (peak load)	Yes	OPT imitation	Empirical
FairyWREN (OSDI’24) [?]	Object	TCO (writes $\rightarrow$ lifetime / cost)	No	Heuristic	Analytical
SeqPack (JSA’23) [?]	Segment	Latency	Yes	Heuristic	Analytical
GL-Cache (FAST’23) [?]	Object grp.	Hit rate	No	Sup. ML	None
CacheSack (ATC’22) [?]	Category	TCO (reads + writes + usage)	No	Analytical	Analytical
ETICA (TPDS’21) [?]	Segment / Object	Hit rate / Latency	Yes	Heuristic	Analytical
DACache (HPCA’20) [?]	Object	Hit rate / Latency	Yes	RL	Empirical
TLC-Cache (MSST’19) [?]	Block	Hit rate / Endurance	Yes	Heuristic	Analytical
Flashield (NSDI’19) [?]	Object	Hit rate (WA↓)	No	Sup. ML	Empirical
Multi-Cache (ICS’18) [?]	Segment	Hit rate / Latency	Yes	Heuristic	Analytical
LARC (TOS’16) [?]	Block	Latency	No	Heuristic	None
RIPQ (FAST’15) [?]	Object	Latency / Throughput	No	Heuristic	None
HybridStore (HPCA’13) [?]	Block / Segment	Latency / TCO	Yes	Heuristic	Empirical
FlashTier (EuroSys’12) [?]	Block	Latency / Robustness	No	Heuristic	None

Table 2: Timeline and evidence synthesis of ML-guided caching/prefetching papers.

Period	Key Research Focus	Representative Works	Metrics / Objectives	Limitations Relative to Baleen
2010–2015	OS-managed tier; write layout	<i>FlashTier</i> (EuroSys’12) [?]; <i>HybridStore</i> (HPCA’13) [?]; <i>RIPQ</i> (FAST’15) [?]	Hit rate; latency/robustness; sequentialize writes; group objects for throughput/lifetime	No DT/peak-load cap; admission-prefetch uncoordinated; no endurance/TCO model
2016–2020	Admission filtering; endurance awareness; DRAM+SSD; data-aware placement	<i>LARC</i> (TOS’16) [?]; <i>MultiCache</i> (ICS’18) [?]; <i>Flashield</i> (NSDI’19) [?]; <i>TLC-Cache</i> (MSST’19) [?]; <i>DACache</i> (HPCA’20) [?]	Reduce needless SSD writes; lower WA while keeping hit rate; optimize tiering; place by temperature/frequency to cut I/O stalls	No explicit DT objective; prefetch largely absent/decoupled; endurance/TCO heuristic; several policies static
2021–2025	Cost/TCO-aware + ML; DT-aware coordination; interface isolation	<i>ETICA</i> (TPDS’21) [?]; <i>CacheSack</i> (ATC’22) [?]; <i>SeqPack</i> (JSA’23) [?]; <i>GL-Cache</i> (FAST’23) [?]; <i>Baleen</i> (FAST’24) [?]; <i>FairyWREN</i> (OSDI’24) [?]; <i>FDP-CacheLib</i> (EuroSys’25) [?]	Minimize total cost(reads+writes+usage); batch/seq writes for hybrid; DT-aware ML admission+prefetch; GC co-design; NVMe FDP to curb amp/tails	Except <i>Baleen</i> : admission/prefetch not co-trained; DT not first-class; no global TCO control. <i>Baleen</i> = DT/coordination-aware baseline.

Member Contributions

To start the paper, every team member read and understood the paper. Below are the contributions for each member:

Jun Tang (2111172)

- Read the paper and discussed with team members.
- Add EDE policy class to the code.
- Run the experiment and get the data
- Draw the figure 3 and write the result, discussion and analysis for figure 3.
- Team coordination - set up latex in Overleaf, kanban project plan.

Ziyan Qiu (2089937)

- Generated and formatted Figure 4 (Peak DT vs. α_{tti}) for the E2 ablation study.
- Wrote the corresponding Results, Analysis, and Limitations sections for Figure 4.
- Helped keep LaTeX formatting consistent across all figures and captions.
- Integrated all sections into the final Evaluation report and verified ACM formatting.
- Reviewed the full draft and helped with final proofreading before submission.

Kaiweng Wang (2073516)

- Created and analyzed Figure 5
- Aggregated and normalized sensitivity data across τ_{DT} , PROTECTED cap, and α_{tti} parameters.
- Integrated the finalized figure into the Evaluation section and ensured layout consistency with the team's report.
- Wrote the Results, Discussion, and Limitations sections for Figure 5 in LaTeX.
- Adjusted document formatting and verified consistency across captions, and section titles.

Xiaoyi Han (2133542)

- Created and analyzed Figure 4.
- Ran simulations for LRU and E2
- Collected experiment data.
- Plotted Peak DT vs. Cache Size results.
- Wrote analysis and discussion section.

Hongyi Niu (2104640)

- Read the paper and discussed with team members.
- Created and analyzed Figure 1.
- Ran multiple DT-SLRU experiments with different values.
- Collected and cleaned the simulation results for Peak Disk-head Time.
- Wrote the Results, Discussion, and Limitations sections for Figure 1.

References